
파일 시스템과 I/O의 내부 동작 원리

File Descriptor, File System
I-Node와 BufferedReader

1. 파일 시스템 (+ 파일 시스템 없애는 법)
2. I-Node란 (+ Hard Link vs. Symbolic Link)
3. File Descriptor
4. 시스템 콜 & 추상화

File System

데이터는 어디에 저장될까?

- 하드 디스크라는 거대한 데이터 창고에 저장됨.

파일 시스템이 없다면?



파일 시스템 적용



데이터의 지도, 파일 시스템 (File System)

하드디스크라는 데이터 창고에
‘질서’를 부여하는 역할!

파일 시스템의 역할

1. **계층적 구조**
폴더(디렉터리) 구조를 통해 파일을 체계적으로 정리
Ex) C:\Users\SSAFY\...
2. **이름 부여**
각 파일과 디렉터리에 고유한 이름을 붙일 수 있다.
3. **데이터 저장 및 검색**
파일의 실제 데이터가 디스크의 어느 물리적 위치에 저장되어 있는지 기억하고, 필요할 때 빠르게 찾아 준다.
4. **메타데이터 관리**
파일의 크기, 생성일, 수정일, 소유자, 접근 권한 등
파일의 내용이 아닌 '파일에 대한 정보'를 관리
5. **접근 제어**
특정 사용자나 그룹만 파일을 읽거나 쓸 수 있도록 권한을 관리

운영체제별 파일 시스템 구조

리눅스 (macOS 등 Unix 계열)

- 단일 루트 (Single Root) 트리 구조
- 모든 파일은 최상위 디렉터리 “/”에서 시작
- C드라이브, D드라이브 같은 개념X
→ 새로운 하드디스크나 USB가 연결되면,
/mnt나 /media같은 기존 트리의 디렉터리에
마운트 해서 사용

윈도우 (Windows)

- 다중 루트(Multiple Roots) 구조
- C:\, D:\처럼 각 드라이브가 독립적인 루트
- 파일 시스템 전체의 최상위 루트 (/) 가 없음

File System

파일 시스템을 파괴하는 명령어 `sudo rm -rf(linux)`



```
sudo rm -rf /
```

진정 새로운 무언가를 창조하려 한다면 처음부터 다시 시작해야 합니다.

File System

파일 시스템을 파괴하는 명령어 `sudo rm -rf(linux)`



파일 시스템을 파괴하는 명령어 `sudo rm -rf(linux)`

- `sudo`: 'Substitute User and do' 의 약자, 명령어를 최고 관리자 권한으로 실행
- `rm`: 'remove'의 약자. 파일을 지우라는 명령어입니다.
- `-r`: 'recursive'(재귀적인)의 약자. "만약 지우려는 게 디렉터리라면, 그 안에 있는 모든 파일과 하위 디렉터리를 전부 다 지워라"는 옵션입니다. (폴더 하나를 지우면 그 안의 내용물이 전부 삭제되는 게 바로 이 `-r` 옵션 때문)
- `-f`: 'force'의 약자. "강제로 지워라"는 뜻입니다. '정말 지우시겠습니까? (y/n)' 같은 확인 절차를 전부 무시하고, 권한이 허용하는 한 모든 걸 삭제
- `/`: 루트 디렉터리(root directory). 우리가 배운 파일 시스템의 최상위 꼭짓점, 모든 것의 시작점입니다.

File System

파일 시스템을 파괴하는 명령어 `sudo rm -rf(linux)`

- 실행하면?
- 명령어는 파일 시스템의 정점인 루트(/)에서 시작해서 자기 자신(명령어)를 포함한 Os, 파일 시스템에 포함된 모든 것이 삭제

파일 시스템을 파괴하는 명령어 `sudo rm -rf(linux)`

- 생생한 후기

❤️됐다❤️발 어카냐

o o (27.35) | 2021.01.15 21:37:47

외주업체에 cms모듈 만들어서 외주업체 서버에 설치하려 했는데
서버는 CentOS인데 파일지울일이 있어서
rm을 썼음 근데 파일이 루트폴더쪽에 있어서 권한오류뜨면서 안지워지더라
그래서 sudo 붙여서 지우려했는데

아뿔싸

`sudo rm -rf ./*` 이렇게 쳐야될거를
`sudo rm -rf /*` 이렇게 쳐버림

❤️발 지금은 ssh접속도 안된다
어카냐 시❤️

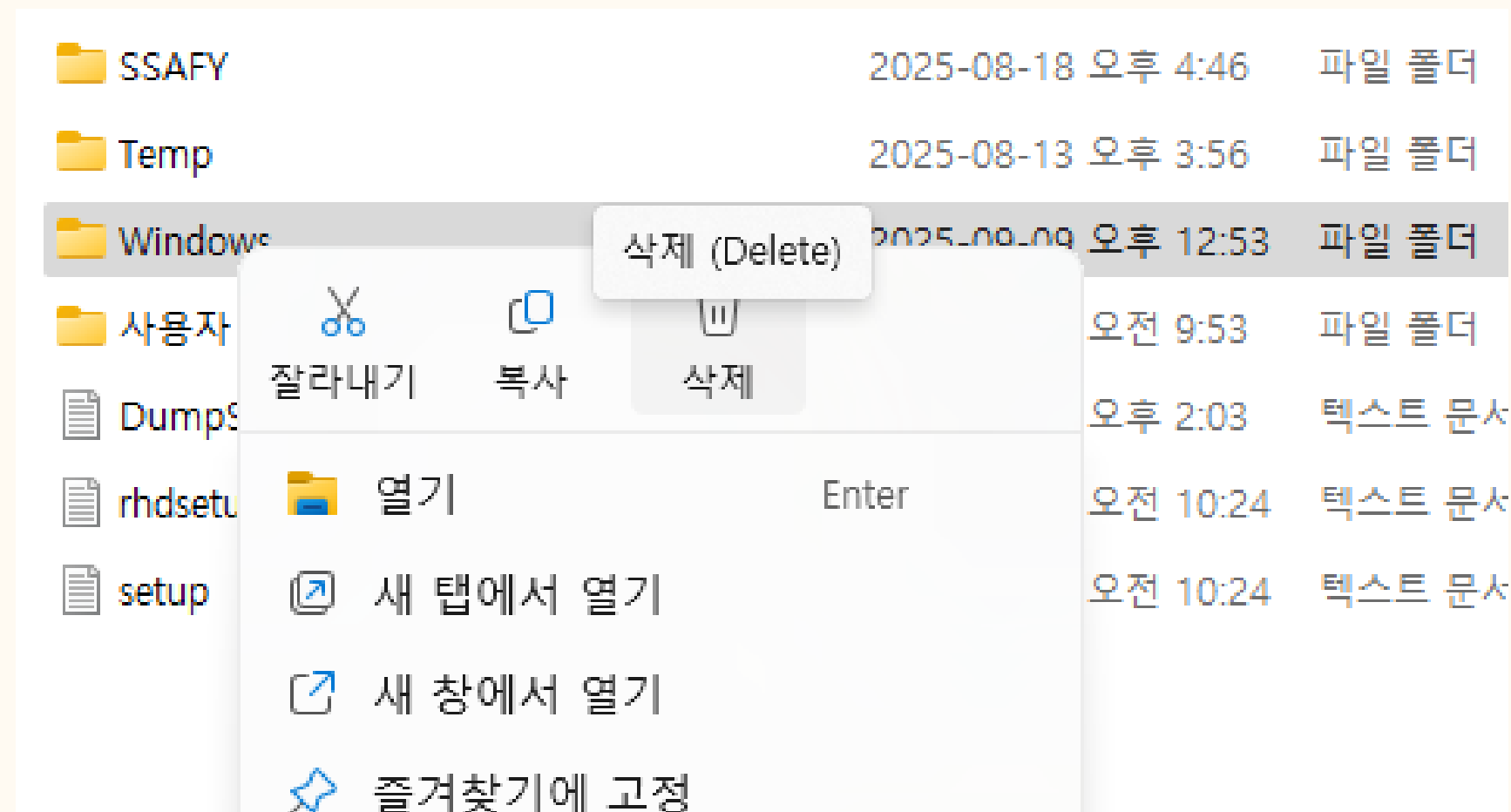
File System

파일 시스템을 파괴하는 명령어 `sudo rm -rf(linux)`

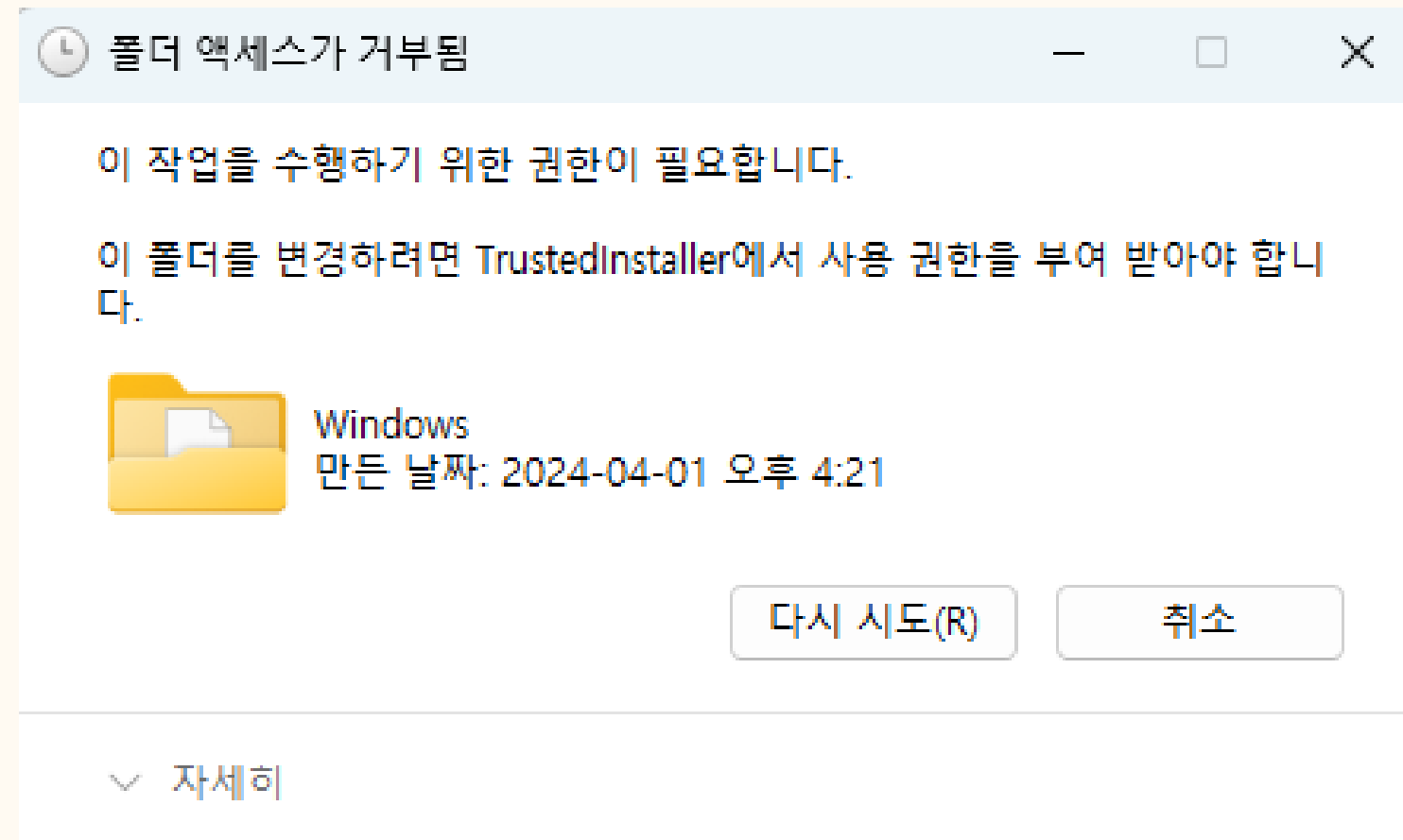
- 윈도우에서는?
- `rd /s /q C:\`를 통해서, C:\ 하위에 있는 모든 파일 시스템 삭제(OS포함)
(`rd /s /q D:\` → D드라이브 삭제)

File System

- 윈도우 파일 탐색기에서 삭제를 시도해 본다면?
- 서울 7반 29번 좌석의 운영 체제를 삭제해보자



File System



아쉽게도 실패한다! why?

파일 시스템을 파괴하는 명령어 `sudo rm -rf(rinux)`

- TrustedInstaller: 시스템 계정 (관리자 계정보다 상위)
- GUI (파일 탐색기) 에서 핵심 시스템 파일들을 실수로 삭제하는 것을 방지하기 위해서, 이러한 작업들을 실시할 경우 권한 문제로 막힌다. (안전장치)
- 그러나, CLI(관리자 권한 터미널)에서는 훨씬 낮은 레벨에서 OS와 직접 소통하고, 높은 권한을 갖고 있음. 따라서 `rd /s /q C:\`와 같은 명령어가 실행 가능!

파일 시스템은 파일을 어떻게 관리할까?

I -Node(index - node):
파일 각각에 대한 모든 메타데이터 담고 있는 정보 덩어리

I – Node에 포함된 정보

- 아이노드 번호
- 파일의 종류(일반 파일? 디렉터리? 심볼릭 링크 등)
- 접근 권한 (읽기, 쓰기, 실행 권한)
- 소유자(UID) 및 그룹(GID)
- 파일 크기(bytes)와 타임스탬프(생성, 접근, 수정한 시각)
- **데이터 블록 포인터: 파일의 실제 데이터가 저장된 디스크 상의 위치**(주소)를 가리키는 포인터들

Hard Link vs Symbolic Link

- 파일의 실체는 I-Node에 저장된 메타데이터와 데이터 블록 포인터
- 우리가 붙이는 파일의 이름(myfile.txt)과 I - Node(실체)는 어떻게 연결되는가

→ hard link와 symbolic link 으로 나뉜다.

I-Node

Hard Link

- 하나의 I-Node에 여러 파일 이름을 연결
- 원본과 사본의 개념 X, 둘 다 원본 (동일한 I-Node 번호를 공유)
- I-Node 내부의 링크 카운트(Link Count)가 0이 되기 전까지는 I-Node는 삭제X
- 예) 파일을 생성하면 → I-Node와 첫 번째 하드 링크가 생성(I-Node 내부의 Link Counter = 1)
- In original.txt hardlink.txt 를 통해 하드 링크를 만들면, 두 파일명은 동일한 I-Node를 가리킨다 (자바에서 변수와 객체의 관계)

Symbolic Link

- 심볼릭 링크 파일은 원본과 전혀 다른 I-Node를 가짐
- 원본 파일을 가리키는 ‘포인터 역할’을 하는 특수한 파일
- 원본을 삭제 시, 링크가 끊어짐(Broken Link)
- 윈도우의 ‘바로 가기’와 거의 똑같은 개념
- 심볼릭 링크를 지우는 건, 원본 파일에 아무런 영향 X

I-Node

Hard Link

- 디렉터리는 링크가 불가능하다
- Why? 순환 참조가 발생한다.
- 예시) A 폴더 밑에 B 폴더
- B 폴더 안에 C라는 이름의 A를 가르키는 하드 링크를 만드는 게 가능하다면?
- /A와 /A/B/C는 완전히 동일한 I-Node를 공유
- $A \rightarrow B \rightarrow C \rightarrow B \rightarrow A \rightarrow B \dots$ 의 무한 루프 발생
- 다른 파일 시스템(파티션)으로는 링크 불가 (I-Node 번호는 해당 파일 시스템 내에서 고유함)

Symbolic Link

- 디렉터리도 링크 가능하다.
- 다른 파일 시스템(파티션)으로 링크 가능

File Descriptor, FD

프로세스가 파일에 접근하기 위해서는 어떻게 해야할까?

File Descriptor(FD) : 실행 중인 프로세스가 특정 파일에 접근하기 위해서 커널로부터 부여받는 고유한 정수(Integer) 값

File Descriptor, FD

FD의 동작 방식

1. 프로세스가 `open("/home/ssafy/my_file.txt")` 시스템 콜을 호출
2. 커널은 파일 시스템을 계층적으로 탐색해서 `my_file.txt`의 아이노드 번호를 찾는다.
(/ root의 번호 (보통 2번) → home의 번호 → ssafy의 번호 → `my_file.txt`의 번호)
3. 커널은 이 프로세스의 '파일 디스크립터 테이블'에 새로운 항목을 만들고, 이 항목이 파일의 아이노드를 가리키게 함.
4. 커널은 이 테이블의 **인덱스 번호**(정수)를 프로세스에게 돌려준다.
이 번호가 바로 파일 디스크립터
5. 프로세스는 이제부터 파일에 데이터를 읽거나 쓸 때(`read()`, `write()`) 긴 파일 경로를 쓰는 게 아니라, 이 파일 디스크립터 번호만 커널에 넘겨준다.

File Descriptor, FD

기본 파일 디스크립터

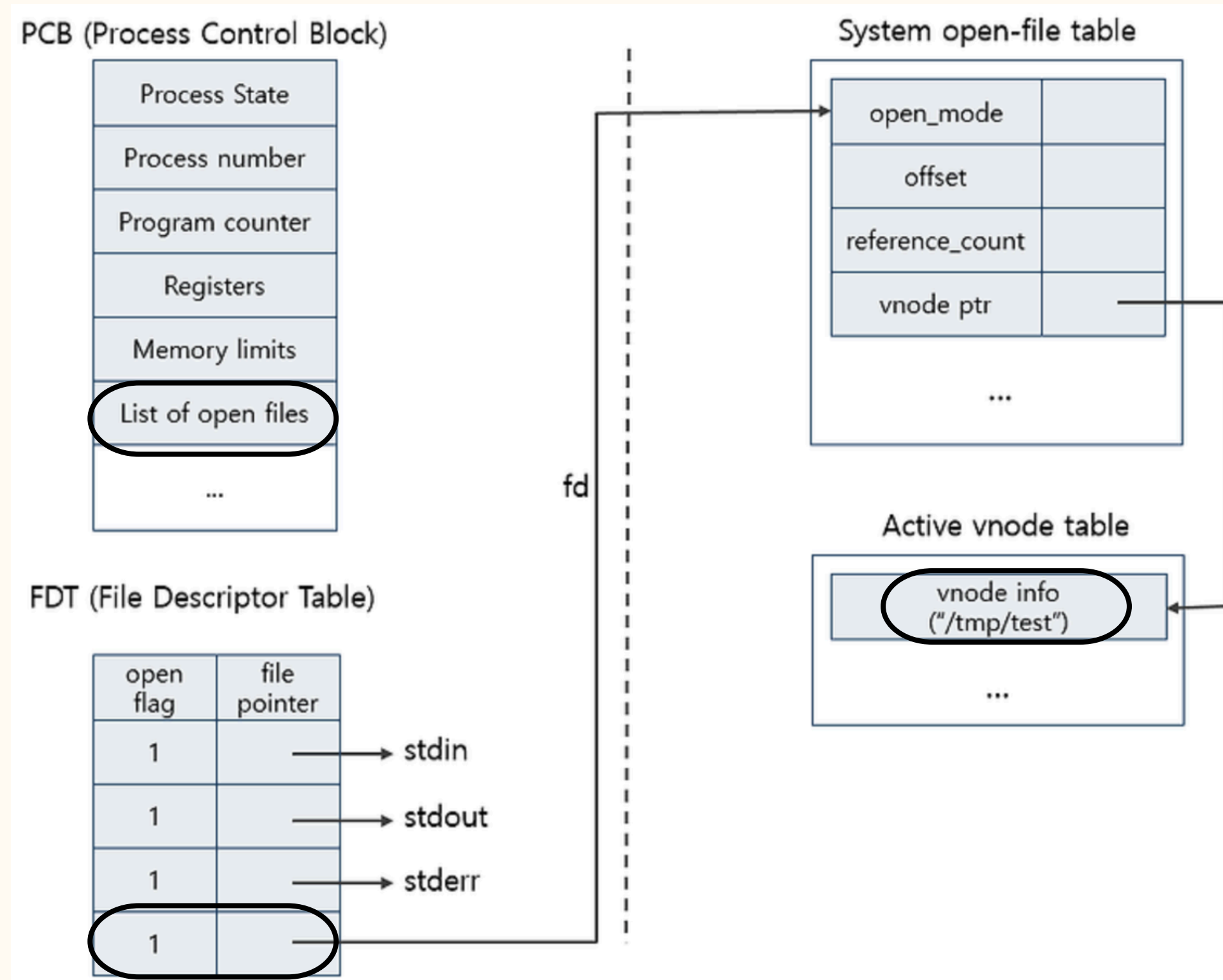
- 모든 프로세스는 실행될 때, 기본적으로 3개의 파일 디스크립터를 할당받음
- 0: 표준 입력 (Standard Input, stdin) – 키보드
- 1: 표준 출력 (Standard Output, stdout) – 모니터
- 2: 표준 에러 (Standard Error, stderr) – 모니터

코드와 OS를 이어주는 시스템 콜과 추상화

- Python의 `open()`이나, 자바의 `BufferedReader`의 작동 방식
`BufferedReader br = new BufferedReader(new FileReader("my_file.txt"));`
1. JVM(자바 가상 머신)이 OS에게 `open("my_file.txt")` 시스템 콜을 요청
 2. OS 커널이 파일 시스템을 뒤져서 `my_file.txt`와 연결된 `I-Node`를 찾아냄
 3. 이 프로세스의 `FD`테이블에 `my_file.txt`와 연결된 인덱스(`FD`)를 할당 (ex. 3)
 4. OS는 방금 할당한 3을 JVM에게 돌려줌.(이 인덱스는 `FileInputStream` 객체 내부에 존재, 외부 노출 X)
 5. `br.readLine()`을 호출하면, `br` 객체 내부의 `FD`를 꺼내서 OS에게 `read()` 시스템 콜을 요청
 6. `br.close()`를 호출하면, JVM은 `FD`를 반납하는 `close()` 시스템 콜을 OS에 요청

Java 코드 (Application) → JVM → 시스템 콜 → OS 커널 (파일 시스템, I-Node, FD 관리) → 하드웨어 (디스크)

시스템 콜 & 추상화



감사합니다!